

Oxy Comprehensive Overview – May 2026

Generated from:

- `/Users/chizigamonyewuchi/Desktop/oxy-state-may-2026.md`
- The live codebase in `/Users/chizigamonyewuchi/Documents/Oxy`

Principle used in this document: where the May state doc and the codebase differ, the codebase is treated as the current implementation truth. The May state doc remains the source for product vision, hardware, brand, manufacturing, and business intent where those areas are not represented in code.

1. What Changed From The May State Doc

This is the most important section because the May doc is slightly behind the repo in several areas.

1.1 Pricing Changed

The May doc says:

- Pendant: £349
- Kickstarter early bird: £299
- Subscription: £24.99/month or £179/year
- First month free

Updated pricing direction:

- Pendant remains positioned around £349 unless deliberately changed later.
- Kickstarter early bird remains sensible at £299.
- Subscription must now be treated as maximum £15/month.
- Annual pricing should be recalculated from the new ceiling. A clean option is £149/year or lower.
- Any subscription margin, milestone, and investor narrative based on £24.99/month is now outdated.

Implications:

- Oxy is more accessible and easier to justify as a daily life assistant.
- The subscription story becomes "premium but not painful" instead of high SaaS pricing.
- API-cost discipline matters more. Gemini Live, caching, shorter speech, deterministic routing, and future on-device intent classification become financially central.
- At £15/month, blended net revenue after App Store/web mix is materially lower than the May model. The product can still work, but usage caps, efficient model routing, and a strong annual plan matter.

Suggested new subscription model:

Plan	Price	Notes
---	---	---
Oxy Core	Included / free trial	Basic setup, limited assistant use, onboarding, connector demos

Oxy Plus	£14.99/month max	Main subscription; voice, memory, proactive briefings, connectors
Annual	£129-£149/year	Keeps monthly equivalent below £12.50
Founder / Kickstarter	3-12 months included	Useful for conversion and launch momentum

1.2 Proactiveness Is No Longer Just "Broken"

The May doc says proactive triggers are written but broken because serverless kills the interval.

The repo now includes a proper direction:

- `proactive-job.js` runs `runProactiveSweep`.
- `npm run proactive:job` exists in `package.json`.
- README documents deploying a separate Cloud Run Job scheduled every 15 minutes.
- API endpoints now exist for:
 - `POST /proactive/:userId/run`
 - `ALL /proactive/sweep`
 - `GET /briefings/:userId`
 - `POST /briefings/:id/read`

Current state:

- Proactive system is implemented as backend logic and has a Cloud Run Job path.
- It still depends on deployment setup, env vars, device registration, native context, and scheduler configuration.
- It is better described as "implemented, needs deployment verification" rather than "broken."

1.3 The iOS App Is No Longer Purely Future-Tense

The May doc says the iOS companion app is blocked until the Mac arrives.

The repo now contains a SwiftUI native app under `OxyApp/OxyApp` with:

- Auth/login/register
- Chat UI
- Streaming SSE chat
- Image chat
- Connectors view
- Memory view
- Action history
- Proactive briefings tab
- Settings
- Native integrations for location, contacts, HealthKit, Reminders, Calendar, MusicKit/Apple Music, MessageUI, FaceTime, and local place search
- Keychain-backed session storage
- APNs/device registration plumbing

Current state:

- The native app exists as code.
- It appears to be a serious companion-app implementation, not merely a placeholder.
- Hardware BLE bridging is not present in the inspected files.

- Dynamic Island / Live Activities are not visibly implemented in the inspected code.
- Accessibility API control is not visibly implemented in the inspected code.

1.4 Gemini Live Is More Than A Plan

The May doc describes Gemini Live migration as in progress.

The repo now has two WebSocket voice paths:

- `/realtime-voice` in `live.js`
- `/companion-live` in `companion-live.js`

There is also a benchmark script:

- `scripts/benchmark-voice-pipelines.js`

Current state:

- The current PWA path still supports upload/SSE voice through `/process-audio`.
- The chat path supports streaming text and sentence-level TTS chunks.
- Gemini Live is present as a realtime WebSocket implementation and companion prototype.
- Full product migration still needs practical verification across devices.

1.5 Connector Set Expanded

The May doc lists live connectors as Gmail, Google Calendar, Uber, Telegram, Trainline, with Trainline unreliable.

The repo's implemented connector registry now includes:

- Google: Gmail + Calendar
- Uber
- Uber Eats
- Deliveroo
- Netflix
- Telegram
- Trainline
- Maps

The UI-visible connector list in the current backend is:

- Google
- Netflix
- Deliveroo
- Uber Eats
- Uber
- Maps
- Telegram
- Trainline

Important distinction:

- Some are true API integrations with stored tokens.
- Some are app-opening/deep-link connectors.
- Some degrade gracefully to web links when APIs are unavailable.

1.6 Action Safety Is More Mature

The May doc talks about action execution generally.

The repo now has explicit action contracts:

- Required fields
- Risk levels
- Confirmation/review mode
- Execution mode
- Recovery metadata
- Connector health diagnosis
- Pending review flow

High-risk actions such as sending email, Telegram messages, calls, and food orders are review-gated rather than blindly executed.

1.7 Product Is Now Multimodal And Creative

The May doc focuses on voice assistant, memory, and phone control.

The repo also supports:

- Image uploads for chat
- Image generation
- Diagram artifact generation
- Presentation outline/deck artifact generation

These are implemented as internal/native creative tools in ``api/index.js`` and exposed to the model through the system prompt.

1.8 Auth Is Now Per-User

The README and code show that Oxy now uses:

- User registration
- User login
- Password hashing with ``crypto.scryptSync``
- Signed session tokens
- Per-user Supabase records
- Auth checks on most API routes

This is meaningfully more production-shaped than a shared API secret.

1.9 MCP Exists, But Is Separate

The repo has ``mcp-server.js``, a standalone Express service on port 3100 with a simple ``POST /tools`` endpoint.

It supports tools like:

- SMS/Telegram messaging
- Reminders
- Calls
- Spotify playback
- Calendar event creation
- Home Assistant control

This is not the same as the main app connector architecture. It should be described as adjacent infrastructure / external-agent bridge, not the central product path.

2. Product Definition

Oxy is a wearable AI assistant designed to let a person press a pendant, speak naturally, and have things happen without opening their phone.

Core idea:

- A pendant around the neck.
- Press to talk.
- Oxy listens, understands, remembers, and executes.
- It uses the phone and cloud as the heavy compute/network layer.
- It is positioned as a life operating system, not a chatbot.

What Oxy is:

- A wearable personal assistant.
- A hands-free and screen-light interface for daily life.
- A connector/action system for real services.
- A memory system that learns user context over time.
- A fashion-tech object with swappable shell identity.
- A companion app plus backend today, with pendant hardware as the full product expression.

What Oxy is not:

- Not a phone replacement.
- Not only a note taker.
- Not a coding assistant.
- Not a generic ChatGPT wrapper.
- Not a passive surveillance pendant.
- Not a camera-first product.

Positioning:

- "Where fashion meets intelligence."
- "One intelligence. Infinite expression."

Strategic thesis:

Everyday AI becomes truly useful when it can do three things at once:

- Be present in the moment through voice.
- Know the user through memory and native context.
- Act through real connected services.

3. The Moat

The May doc identifies six moat layers. They still hold, with some updated implementation evidence.

3.1 Hardware

The hardware vision is a titanium pendant with:

- 38mm circular form factor.
- Press-to-talk interaction.
- Bone conduction output.
- BLE phone bridge.
- Swappable shell system.
- Jewelry-box charger.

Current codebase status:

- Hardware firmware is not present in this repo.
- BLE bridge code is not visible in the Swift app yet.
- Hardware remains a product/roadmap area rather than a current code implementation.

3.2 Media And Brand

The product depends heavily on visual desirability:

- Fashion object first impression.
- Editorial renders.
- Shell drops.
- Designer collaborations.
- Jewelry-box unboxing.

Current codebase status:

- App visual language exists in PWA and SwiftUI.
- Product renders/assets described in the May doc are not part of this repo except app icons and connector icons.

3.3 Deep Tech

Implemented or partially implemented:

- Gemini chat.
- Gemini STT.
- Gemini TTS.
- Gemini Live WebSockets.
- Memory extraction.
- Prompt caching.
- Context caching.
- Deterministic intent routing.
- Action contracts.
- Connector health recovery.
- Native context ingestion.

Future:

- On-device intent classification.
- Offline commands.
- Local action triage on the pendant/phone.

3.4 UX

Implemented:

- PWA chat, memory, settings, connectors, history.
- SwiftUI native chat and tabs.
- Streaming responses.
- Voice playback.

- Action cards.
- Review sheets.
- Deep-link opening.
- Briefing feed.

Planned / not visibly implemented:

- Dynamic Island / Live Activities.
- BLE pendant control.
- Accessibility API phone control.
- Finished jewelry-box UX.

3.5 Trust

Trust posture:

- Press-to-talk is central to the brand.
- No camera in hardware plan.
- User memory is visible/editable.
- High-risk actions are review-gated.
- Per-user auth exists.
- Connector tokens are stored per user in Supabase.

Still needs:

- Public privacy policy.
- Data deletion/export UX beyond current memory deletion.
- Token encryption strategy beyond database storage.
- Security review before real users.

3.6 High-Agency AI

Implemented:

- Gmail send/read/search.
- Calendar read/create.
- Telegram send/contacts.
- Uber links.
- Uber Eats links.
- Deliveroo links.
- Netflix links.
- Maps place/directions/trip planning.
- Trainline search/station board.
- Native iOS actions for messages, calls, reminders, calendar, music, health answers, contacts, places.

Planned / partial:

- Accessibility API app control.
- More reliable commerce booking.
- Richer native app automations.
- Broader connector ecosystem.

4. Hardware Overview

The hardware vision is taken from the May state doc because the repo does not contain firmware or CAD.

4.1 Form Factor

- 38mm circular pendant.
- Worn around the neck.
- Integrated bail.
- Chain included.
- Core + shell architecture.

The core contains:

- MCU.
- Microphone.
- Battery.
- Haptics.
- Bone conduction driver.
- Touch/press input.
- Charging.
- IMU/health sensors depending on version.

The shell provides:

- Fashion identity.
- Material finish.
- Swappable expression.
- Drop/collab revenue.

4.2 Materials

Target shell:

- Grade 5 titanium.

Reasons:

- Lighter than stainless steel.
- Premium feel.
- Hypoallergenic.
- Strong scratch resistance.
- Better vibration conduction for bone conduction.
- Supports premium pricing.

Face:

- Frosted Gorilla Glass 6.

4.3 Battery And Connectivity

Battery:

- May doc target: 400-500mAh.

Connectivity:

- BLE-only pendant.
- Phone handles internet.
- Backend handles AI and services.

Intended path:

Pendant -> BLE -> iOS companion app -> WiFi/5G -> Oxy backend -> connectors/Gemini -> response -> phone -> BLE -> pendant.

4.4 Chips

Prototype:

- ESP32-S3.

Production target:

- nRF52840 via Raytac MDBT50Q-512K.

Reasoning:

- ESP32-S3 is easier for prototyping and possible TensorFlow Lite Micro experiments.
- nRF52840 is more suitable for BLE wearable power use.

4.5 Bone Conduction

V1:

- Transducer on back of pendant.
- Vibrates through titanium shell into collarbone.

V2:

- Transducer integrated into chain at neck.
- Better acoustic path.
- Requires more complex mechanical/electrical design.

Key blocker:

- Needs physical testing. The May doc is clear that V1 must be tested against the collarbone before committing.

4.6 Charging

Vision:

- Jewelry box is the charger.
- Qi wireless charging.
- Pendant sits in custom recess.
- Portable enough for travel.

This is strategically important because it makes the object feel like jewelry, not a gadget.

4.7 Locked BOM From May Doc

Component	Part Number / Target
---	---
Production MCU	Raytac MDBT50Q-512K / nRF52840
Dev MCU	ESP32-S3
Microphone	Knowles SPH0645LM4H
Bone conduction transducer	AAC Technologies AT23B46
Bone conduction amp	TI TPA6205
Charging IC	TI BQ25185
Touch IC	Infineon CY8CMBR3102
IMU	Bosch BMI270
Heart rate	Maxim MAX30102
LED	Worldsemi WS2812B SMD 5050
Haptic motor	AAC Technologies NFP-ELV1040
Qi module	TDK WR202

| Battery | ATL 503040 500mAh |

4.8 Manufacturing

PCB:

- 4-layer FR4.
- 38mm circular.
- 1.6mm thickness.
- Black soldermask.
- ENIG finish.
- JLCPCB target.

Shell:

- CNC titanium.
- Xometry or Shenzhen supplier.
- One sample before batch.

Missing:

- Gerbers.
- PCBA quote.
- CNC quote.
- Acoustic test data.
- Thermal/power measurements.
- Waterproofing plan.
- DFM review.

5. Software Architecture

Oxy is currently a hybrid product:

- Node/Express backend.
- PWA frontend.
- Native SwiftUI iOS app.
- Supabase database.
- Gemini model layer.
- Connector/action system.
- Realtime voice prototypes.
- Standalone MCP server.

5.1 Runtime Stack

Backend:

- Node.js.
- Express 5.1.
- WebSocket via `ws`.
- Multer for file uploads.
- Axios for external APIs.
- Supabase JS client.
- Google Gemini SDKs:
 - `@google/generative-ai`
 - `@google/genai`

Frontend PWA:

- Single `index.html`.
- React 18 from CDN.
- Babel in browser.
- Service worker.
- PWA manifest.

iOS:

- SwiftUI.
- Observation.
- URLSession.
- SSE parsing.
- Keychain.
- CoreLocation.
- HealthKit.
- Contacts.
- EventKit/Reminders.
- MusicKit/MediaPlayer.
- MessageUI.
- AVFoundation.

Deploy:

- Cloud Run primary target.
- Dockerfile included.
- Cloud Run Job intended for proactive sweeps.

5.2 Main Server Entry

`server.js`:

- Imports the Express app from `api/index.js`.
- Creates an HTTP server.
- Attaches `/realtime-voice`.
- Attaches `/companion-live`.
- Listens on `0.0.0.0:\$PORT`, default 3000.

5.3 Required Environment Variables

Required by `runtime.js`:

- `SUPABASE\_URL`
- `SUPABASE\_KEY`
- `GEMINI\_API\_KEY`
- `OXY\_SESSION\_SECRET`

Optional:

- `TIMEZONE`
- `APP\_URL`
- Google OAuth env vars.
- Telegram env vars.
- TransportAPI env vars.
- Google Maps / Places env vars.
- APNs env vars.
- Twilio env vars.
- Spotify env vars.
- Home Assistant env vars.
- Vertex/enterprise Gemini env vars.

6. Backend API Surface

Most routes require session auth. Public routes include:

- `/`
- `/health`
- `/install-shortcut`
- `/auth/register`
- `/auth/login`
- `/auth/google/callback`

6.1 Auth

Routes:

- `POST /auth/register`
- `POST /auth/login`

Implementation:

- User IDs must match `/^[a-zA-Z0-9\_-]{1,128}\$/`.
- Password must be 8-1024 characters on register.
- Passwords are hashed using Node crypto script with a random salt.
- Session token is signed with HMAC SHA-256.
- Session TTL is 7 days.
- Auth accepted through `Authorization: Bearer ...` or `X-Session-Token`.

6.2 Chat And Voice

Routes:

- `POST /chat`
- `POST /process-audio`
- `POST /chat-with-image`
- `POST /tts-preview`

`/chat`:

- Supports JSON chat.
- Supports streaming via SSE when `stream=true`.
- Supports TTS when `tts=true`.
- Loads memory, preferences, history, connector context, location/native hints.
- Uses deterministic quick replies and deterministic routing for some local/travel requests.
- Uses Gemini for general reasoning.
- Parses `` blocks.
- Executes actions through the action runner.
- Streams status events, text chunks, replacement text, actions, audio chunks, and done events.

`/process-audio`:

- Accepts uploaded audio.
- Rate-limited to 10 requests per minute per user.
- Transcribes with Gemini.
- Runs the main chat pipeline.
- Generates TTS.

- Streams SSE events.

`/chat-with-image`:

- Accepts image upload.
- Sends image to Gemini with message context.
- Can execute resulting actions.
- Can return TTS.

`/tts-preview`:

- Generates a short voice preview.

6.3 Images And Creative Artifacts

Routes:

- `POST /images/generate`
- `POST /chat-with-image`

Internal actions:

- `generate\_visual`
- `create\_diagram`
- `create\_presentation`

Current creative capabilities:

- Generate a visual image from prompt and optional reference image.
- Analyze image context in chat.
- Create a diagram artifact with Mermaid plus generated preview image.
- Create a presentation outline/deck artifact with slide structure and generated cover image.

6.4 Memory

Routes:

- `POST /memory`
- `GET /memory/:userId`
- `DELETE /memory/:userId`

Memory behavior:

- User can manually add memory.
- Oxy extracts a short personal fact after interactions.
- Memory is injected into the system prompt.
- Memory is treated as background context, not something to surface randomly.
- `forget\_memory` exists as an action.

Memory sources:

- `fact`
- `manual\_profile`
- Other action-derived sources as inserted by code.

6.5 Preferences

Routes:

- `GET /preferences/:userId`
- `DELETE /preferences/:userId`

Preference behavior:

- Oxy can learn style/preferences from user instructions.
- Preferences are stored in Supabase.
- Preferences are injected into prompt context.
- Some proactive state is also stored as preferences.

6.6 Connectors

Routes:

- `GET /connectors/:userId`
- `POST /connectors`
- Google OAuth:
 - `GET /auth/google/redirect-uri`
 - `GET /auth/google/start`
 - `GET /auth/google/callback`
- Telegram auth:
 - `POST /auth/telegram/start`
 - `POST /auth/telegram/verify`
 - `POST /auth/telegram/2fa`

Connector UI states:

- `available`
- `connected`
- `needs\_reconnect`
- `needs\_setup`
- `degraded`

6.7 Actions

Routes:

- `POST /action-log`
- `GET /action-log/:userId`
- `GET /action-contracts`

Action system:

- Model returns structured action blocks.
- Actions are validated against contracts.
- Missing fields create recoverable errors.
- High-risk actions can enter pending review.
- User can confirm/cancel/revise pending actions.
- Action results are logged.
- Connector problems are diagnosed into recovery metadata.

6.8 Native And Devices

Routes:

- `POST /devices/register`
- `POST /native/context`
- `POST /native/local-action`

Native context includes:

- Location.
- Health.
- Capabilities.
- Settings.

Native local actions:

- iOS can execute some requests locally.
- The result is logged to the backend so Oxy memory/history stays coherent.

6.9 Proactive

Routes:

- `GET /briefings/:userId`
- `POST /briefings/:id/read`
- `POST /proactive/:userId/run`
- `ALL /proactive/sweep`
- `GET /briefing/:userId`
- `GET /briefing-legacy/:userId`

Proactive system can create:

- Wake briefing.
- Midday briefing.
- Evening briefing.
- Health alert for unusually low heart rate.
- Location food reminder when near home in evening.
- Follow-up for failed actions that need reconnection or permission.

Proactive windows:

- Wake: 6-10.
- Midday: 12-14.
- Evening: 17-20.

Autonomy gating:

- Low/Quiet settings suppress interval briefings.
- Food reminder needs Active/Bold/High style autonomy.
- Health alerts require `healthAlerts`.
- Location reminders require `locationReminders`.

6.10 History

Routes:

- `GET /history/:userId`
- `GET /history/:userId/search`
- `GET /history/:userId/around`
- `GET /history/:userId/date`

History enables:

- Loading latest conversation.
- Searching previous messages.
- Jumping around a point in time.
- Loading a specific date.

6.11 Debug/Health/Install

Routes:

- `GET /debug/:userId`
- `GET /health`
- `GET /install-shortcut`
- `GET /`

``/debug/:userId`` exposes useful implementation flags such as connector state and Gemini key presence, but should be treated carefully in production.

7. AI Model Layer

7.1 Main Models

Defaults in code:

- Primary chat model: ``gemini-3-flash-preview``
- Fast model: ``gemini-3-flash-preview``
- Streaming chat model: fast model unless overridden.
- Gemini Live model configurable via env.
- Image model: ``gemini-2.5-flash-image``.

Env overrides:

- ``OXY_REASONING_MODEL``
- ``GEMINI_MODEL``
- ``OXY_FAST_MODEL``
- ``GEMINI_FAST_MODEL``
- ``OXY_STREAM_MODEL``
- ``OXY_LIVE_MODEL``

7.2 Search Grounding

The code has ``needsSearch`` logic that enables search for:

- Latest/current questions.
- Public figures.
- News.
- Companies.
- Prices.
- Weather/forecast.
- Launch/release/current availability.
- Entity questions likely to have changed.

It avoids search for:

- Messaging.
- Email.
- Calendar.
- Reminders.
- Booking/order commands.
- Music commands.
- Memory operations.
- Personal-context questions.

This matters because Oxy should not hallucinate changing facts.

7.3 Prompt Philosophy

The system prompt tells Oxy:

- The user leads the conversation.

- Memory is background context only.
- Do not randomly surface stored facts.
- Do not repeat already-stated context.
- Do not mention time/date unless needed.
- Use search/tool/context evidence for changing public facts.
- Stop after one confirmation sentence when an action succeeds.

This is good product behavior. It prevents the assistant from sounding like a creepy memory dump.

7.4 Prompt Caching And Context Caching

Implemented:

- Prompt cache warming.
- Prompt cache state map.
- User context cache with 5-minute TTL.
- Cache invalidation after action execution or native context changes.

Reason:

- Reduce latency.
- Reduce repeated Supabase reads.
- Improve voice responsiveness.

7.5 Voice

Current voice paths:

- Upload/SSE path through `/process-audio`.
- Chat SSE path with optional TTS.
- `/realtime-voice` WebSocket path.
- `/companion-live` WebSocket prototype.

TTS:

- Gemini streaming TTS endpoint.
- Multiple prebuilt Gemini voices.
- Default voice: `Aoede`.
- Sentence-level TTS chunking.
- PCM converted to WAV.

Voice settings in clients:

- Voice on/off.
- Voice selection.
- Voice engine selection in PWA between current path and Gemini Live prototype.

7.6 Gemini Live

Implemented paths:

- `/realtime-voice` in `live.js`
- `/companion-live` in `companion-live.js`

Client events include auth, session start, audio append, audio end, and stop.

Server events include:

- Auth/session status.

- Telemetry.
- User transcript.
- Assistant transcript.
- Audio.
- Actions.
- Turn complete.
- Errors.

Current interpretation:

- Gemini Live is real code now.
- It should be benchmarked and tested on device before being declared the primary product path.

8. Action And Connector System

8.1 Action Contracts

Actions are defined in `api/action-contracts.js`.

Each contract can include:

- Risk level.
- Required fields.
- Optional fields.
- Aliases.
- Example input.
- Success summary.
- Failure summary.
- Confirmation policy.
- Execution mode.
- Guidance.

8.2 Implemented Action Types

Messaging / communication:

- `send\_message`
- `make\_call`
- `send\_email`
- `send\_telegram`
- `get\_telegram\_contacts`

Calendar / reminders:

- `create\_reminder`
- `create\_calendar\_event`
- `get\_calendar\_events`

Email:

- `get\_emails`
- `search\_emails`

Travel / location:

- `book\_uber`
- `find\_place`

- `get\_directions`
- `plan\_trip`
- `search\_trains`
- `station\_board`

Food:

- `order\_uber\_eats`
- `order\_deliveroo`

Entertainment:

- `play\_music`
- `add\_to\_music\_playlist`
- `search\_netflix\_title`
- `add\_to\_netflix\_list`

Memory:

- `forget\_memory`

Creative:

- `generate\_visual`
- `create\_diagram`
- `create\_presentation`

8.3 Safety Model

Examples:

- Low-risk data lookups execute directly.
- Food orders are review-required.
- Email send is review-required.
- Telegram send is review-required.
- Calls are review-required.
- Some actions open external apps rather than complete purchase/order fully.

This is the right posture for early Oxy:

- Fast where safe.
- Friction where consequences are real.

8.4 Connector Registry

Current registered connectors:

- `google`
- `uber`
- `ubereats`
- `deliveroo`
- `netflix`
- `telegram`
- `trainline`
- `maps`

The registry maps action names to connector modules.

8.5 Google Connector

Actions:

- `send\_email`

- `get\_emails`
- `search\_emails`
- `create\_calendar\_event`
- `get\_calendar\_events`

Auth:

- OAuth2.
- Tokens stored per user in Supabase.
- Refresh token support.
- If refresh fails due to expired/revoked token, connector is marked disconnected.

Notes:

- Gmail send uses raw RFC-ish email payload.
- Gmail read/search fetches message snippets/metadata.
- Calendar creates/reads events.

8.6 Telegram Connector

Actions:

- `send\_telegram`
- `get\_telegram\_contacts`

Auth:

- Telegram User API.
- Phone auth.
- Code verification.
- Optional 2FA.
- Session stored per user in Supabase.

Notes:

- Sends through user's Telegram session, not bot-only flow.
- Contact matching is search-based and can fail if the contact is not found.

8.7 Uber Connector

Action:

- `book\_uber`

Implementation:

- Builds Uber deep links.
- Uses destination search/geocoding.
- Can use current location for pickup.
- Fallback opens Uber with query rather than direct booking.

Important:

- This is not a full Uber REST booking flow in current code.
- The May doc's "Uber REST API, not deep links" is outdated for current repo behavior.

8.8 Uber Eats Connector

Action:

- `order\_uber\_eats`

Implementation:

- Builds Uber Eats deep link/search URL.
- Review-gated by action contract.

8.9 Deliveroo Connector

Action:

- `order\_deliveroo`

Implementation:

- Builds Deliveroo deep link/search URL.
- Review-gated by action contract.

8.10 Netflix Connector

Actions:

- `search\_netflix\_title`
- `add\_to\_netflix\_list`

Implementation:

- Builds Netflix app/web links.
- Does not appear to use private Netflix account API.

8.11 Trainline Connector

Actions:

- `search\_trains`
- `station\_board`

Implementation:

- Uses TransportAPI where available.
- Builds Trainline booking URLs.
- Has fallback behavior if live rail data is unavailable.

Current status:

- More robust than the May doc implies, but still dependent on TransportAPI permissions and keys.
- The UI marks Trainline degraded if TransportAPI env vars are missing.

8.12 Maps Connector

Actions:

- `find\_place`
- `get\_directions`
- `plan\_trip`

Implementation:

- Uses Google Places/Directions keys when configured.
- Falls back to Apple Maps links when Google setup is missing or fails.
- Supports nearby place ranking using location.
- Supports directions modes.
- Supports rail-first trip planning with Trainline booking URLs.

This is one of the biggest codebase additions beyond the May doc.

9. Frontend PWA

The PWA lives almost entirely in `index.html`.

9.1 Pages

Pages:

- Chat
- History
- Connectors
- Memory
- Settings

9.2 Chat

Features:

- Text chat.
- Hold-to-talk recording.
- Realtime voice WebSocket attempt with fallback to upload flow.
- SSE streaming.
- TTS playback.
- Image attachment/chat.
- Action cards.
- Briefing injection.
- Status labels like Listening, Processing, Online.
- Recent action loading.
- Chat history loading.
- History search/date navigation.

9.3 Connectors

Features:

- Shows connector list from backend.
- Google OAuth popup flow.
- Telegram auth flow.
- Toggle connectors on/off.
- Connector status display.

9.4 Memory

Features:

- View memory summary.
- Add manual memory/profile anchor.
- Clear memory.

9.5 History

Features:

- Action history.
- Action statuses.
- Action results.

9.6 Settings

Features:

- Voice on/off.
- Voice engine selection.
- Voice selection.
- Voice preview.
- Autonomy level.
- Local settings persistence.

9.7 PWA Assets

`manifest.json`:

- Name: Oxy.
- Display: standalone.
- Portrait orientation.
- Categories: social, lifestyle.
- Icons included.

`sw.js`:

- Caches shell assets.
- Network-first for HTML.
- Cache-first for static assets.
- Handles push notifications.
- Avoids caching Supabase/Google API calls.

10. Native iOS App

The native app is in `OxyApp/OxyApp`.

10.1 App Structure

Main tabs:

- Chat
- Proactive
- History
- Connectors
- Memory
- Settings

Core services:

- `APIClient`
- `AuthService`
- `ChatService`
- `LocationManager`
- `NativeIntegrationManager`
- `VoiceInputManager`
- `KeychainHelper`

10.2 Auth And Session

App state:

- Stores user ID and token.
- Restores session.
- Logs out and clears Keychain.

API client:

- Uses `Authorization: Bearer <token>`.
- Clears session on 401.
- Default backend URL currently points to Cloud Run:
 - `https://oxy-151340634966.europe-west2.run.app`

10.3 Chat

Features:

- Sends messages through `/chat`.
- Streams SSE events.
- Plays base64 WAV audio.
- Opens deep links from action results.
- Sends image messages through `/chat-with-image`.
- Loads history.
- Searches chat history.
- Supports action review and message composer handoff.

10.4 Native Local Actions

The native app can intercept some requests and complete them locally before calling the cloud model.

Local/native areas:

- Messages through MessageUI.
- FaceTime/calls.
- Apple Music playback.
- Apple Music add-to-library/playlist.
- Health questions.
- Contact hints.
- Native place search.
- Reminders.
- Calendar events.

The result is logged back to `/native/local-action`.

10.5 Native Context

The app syncs:

- Location.
- Health snapshot.
- Capabilities.
- Settings.

Endpoint:

- `POST /native/context`

This powers proactive briefings and contextual actions.

10.6 Permissions

Declared in `Info.plist`:

- Microphone.
- Speech recognition.
- Location when-in-use.
- Location always.
- Contacts.
- Health.
- Apple Music.
- Calendar.
- Reminders.
- Photo library.

Entitlement:

- HealthKit enabled.

10.7 Proactive View

Features:

- Loads briefings.
- Shows only useful visible briefings.
- Mark as read.
- Manual "check now" path.

10.8 Missing Or Not Visible In Inspected iOS Code

Not found in inspected files:

- BLE pendant connection.
- CoreBluetooth bridge.
- Dynamic Island / Live Activities.
- Accessibility API automation.
- Watch app.
- Production APNs delegate registration beyond app delegate hook and token registration plumbing.

These should remain roadmap items until implemented.

11. Database Schema

Defined in `supabase-migration.sql`.

11.1 Tables

`users`:

- `id`
- `user\_id`
- `password\_hash`
- timestamps

`conversations`:

- user messages and assistant messages.
- role/content/timestamp.

```
`memories`:
- extracted/manual personal facts.
- source.

`action_log`:
- action JSON.
- status.
- error.
- timestamp.

`connectors`:
- connector ID.
- enabled flag.
- tokens JSON.
- per-user uniqueness.

`preferences`:
- key/value user preferences.
- also used for some proactive sent-state keys.

`devices`:
- platform.
- push token.
- timezone.

`native_context`:
- latest location.
- health.
- capabilities.
- settings.

`briefings`:
- proactive briefing feed.
- kind/title/body/source/metadata/read fields.
```

11.2 Database Strengths

- Simple schema.
- Per-user rows.
- Good indexes on user/time.
- JSONB gives flexibility for tokens, action payloads, native context, metadata.

11.3 Database Risks

- Token storage strategy needs hardening before production.
- No row-level security policy shown in migration.
- Service key likely used server-side, which is normal, but production data controls need review.
- Preferences as generic key/value can grow messy without conventions.
- Conversations table may grow quickly and needs retention strategy.

12. Security And Privacy

Implemented:

- Per-user auth.
- Signed session tokens.
- Password hashing.
- Most routes require auth.
- User ID matching prevents accessing another user's data through path params/body.
- CORS can be restricted with `APP\_URL`.
- CSP and HSTS headers are set.
- High-risk action review exists.

Needs attention:

- Token encryption at rest.
- Supabase RLS and service-role boundary.
- Debug endpoint exposure.
- OAuth state/session hardening review.
- Data retention/deletion policy.
- Privacy policy.
- Production logging review, especially around messages and action traces.
- Rate limits beyond audio endpoint.
- Abuse controls for image generation/TTS.
- APNs secret handling.

13. Deployment

Primary target:

- Google Cloud Run.

Main service:

- `server.js`
- port from `\$PORT`.
- Dockerfile included.

Proactive:

- Separate Cloud Run Job.
- `npm run proactive:job`.
- Schedule every 15 minutes with Cloud Scheduler.

Environment:

- `cloudrun.env.example.yaml` exists.
- README documents deployment steps.

Important:

- `npm` was not available in the current Codex shell path during this audit, so automated tests were not run here. The repo does define `npm test` and `npm run smoke`.

14. MCP Server

`mcp-server.js` is a standalone Express app on port 3100.

Tools:

- `send\_message`
- `create\_reminder`
- `get\_reminders`
- `complete\_reminder`
- `make\_call`
- `play\_music`
- `create\_calendar\_event`
- `smart\_home`

Integrations:

- Twilio.
- Telegram Bot API.
- Local JSON reminders file.
- Spotify.
- Google Calendar OAuth refresh credentials.
- Home Assistant.

Interpretation:

- Useful bridge for external agents.
- Not the main app's connector/action system.
- Some actions overlap with main contracts but are implemented separately.
- If kept, it should be aligned with the main connector architecture or clearly documented as experimental/legacy.

15. Business Model

15.1 Hardware Revenue

Current product pricing assumption:

- Pendant: £349.
- Early bird: £299.
- Collector tier can remain around £449 if it includes extra shell(s) and free subscription period.

15.2 Subscription Revenue

Updated cap:

- Maximum £15/month.

Recommended:

- £14.99/month.
- £129-£149/year.
- First month free.
- Kickstarter includes 3 months or more depending on tier.

15.3 Shell Revenue

Launch shells:

- Brushed titanium.
- PVD Noir.
- PVD Gold.

Drop shells:

- Rose gold.
- Seasonal.
- Designer collabs.

Price:

- £79-£89 still plausible.

Strategic role:

- Shells turn hardware into fashion and repeat revenue.
- Drops create cultural moments without requiring new electronics.

15.4 Unit Economics From May Doc

May estimates:

Volume	Unit cost	Margin at £349
100 units	~£240-276	£73-109
1,000 units	~£188	£161
5,000 units	~£145	£204

Status:

- Still estimates.
- Need real PCBA and CNC quotes.
- Subscription price change does not affect hardware margin but affects blended company margin.

15.5 Subscription Economics Need Rework

Old May doc:

- Blended subscription revenue: ~£21.99.
- API cost: ~£5.85.
- Net: ~£16/user/month.

With £14.99/month:

- Web users: near £14.99 before payment fees.
- App Store users: about £10.49 after 30% Apple cut before other costs.
- A 60% web / 40% app blended gross after Apple cut is about £13.19 before payment fees.
- If API costs stayed at £5.85, gross contribution becomes roughly £7.34 before infra/support/payment costs.

Conclusion:

- The subscription can still work, but API cost must be held down.
- Annual web checkout should be pushed strongly.
- Gemini Live latency and cost should be measured, not assumed.
- On-device/local intent routing is financially valuable.

15.6 Updated Revenue Milestones At £14.99

Very rough contribution model using ~£7-£9/month contribution after Apple/API costs:

Subscribers	Rough monthly contribution
---	---
500	£3.5k-£4.5k
1,000	£7k-£9k
5,000	£35k-£45k
10,000	£70k-£90k

This is lower than the May doc. Hardware margin and shell drops therefore matter more.

16. Go-To-Market

The May plan is still coherent:

- Build useful TestFlight app before hardware exists.
- Use proactive iOS assistant as the waitlist hook.
- Convert early software users into Kickstarter backers.
- Launch Kickstarter around September target.
- Demo video must show real-life value with phone untouched.

16.1 Strongest Launch Narrative

"A 20-year-old from Solihull built the wearable assistant Humane tried to make, but as a private, camera-free pendant that actually controls your phone."

Important:

- Avoid sounding like "another AI chatbot."
- Demonstrate action, not conversation.
- Show phone staying in pocket.
- Show fashion object clearly.
- Show privacy: press-to-talk, no camera.

16.2 Best Demo Scenarios

High-conversion demos:

- "Text Mum I am on my way."
- "Find the nearest gym and open directions."
- "Book me an Uber home."
- "What train can I take to London by 9?"
- "Remind me to call Sarah tomorrow."
- "What did I say about my exams?"
- "Summarise my latest emails."
- "Add this song to my playlist."
- "I am near home, remind me to sort food."

16.3 Channels

From May doc:

- Pinterest.

- Reddit.
- Product Hunt.
- Press emails.
- Kickstarter.
- Creator affiliates.
- Shell drops.
- UK jewelry designer collabs.

Additional suggestion from codebase reality:

- TestFlight waitlist should lead with the iOS app capabilities now visible in code: chat, native actions, proactive briefings, connectors, memory.

17. Visual Identity

May doc:

- Warm precision.
- Deep space black.
- Teal accent.
- Off-white text.
- Premium dark mode.

Current codebase:

- PWA uses dark UI with stone/warm neutral accents.
- SwiftUI appears to support themes and accent colors.
- Connector icons are included as asset sets.

Potential mismatch:

- The May doc emphasizes teal; the current app styling leans warm stone/taupe in many places.
- Decide whether teal remains the brand accent or whether the app has evolved toward a warmer jewelry-like palette.

Visual assets still needed:

- Jewelry box charger render.
- Core + shell separated render.
- Shell swap animation/diagram.
- Realistic titanium material studies.
- On-neck scale shots.
- App screenshots.
- Hardware-to-app connection visual.

18. Competitive Landscape

The May competitor table remains useful:

- Taya.
- Nirva.
- Poppy.
- Omi.
- Meta Ray-Ban.
- Clicky.

- Gemini Spark.
- Possible Apple pendant.

Oxy's differentiated combination:

- Pendant form factor.
- No camera.
- Press-to-talk trust model.
- Bone conduction.
- Phone/action control.
- Memory.
- Proactive context.
- Fashion shell system.
- Native companion app.
- Real connector execution.

Risk:

- Big companies can copy software assistants.
- Hardware/fashion/community/brand is the harder-to-copy part.
- Execution speed matters because the product category is moving quickly.

19. Current Working State

Based on the codebase, the following are present:

Backend:

- Express API.
- Auth.
- Chat.
- SSE streaming.
- Gemini integration.
- TTS.
- STT.
- Image chat.
- Image generation.
- Memory.
- Preferences.
- Action contracts.
- Connector registry.
- Proactive logic.
- Cloud Run deployment documentation.

PWA:

- Chat.
- Voice recording.
- Realtime voice attempt/fallback.
- Connectors.
- Memory.
- History.
- Settings.
- Service worker.
- Push notification handling.

iOS:

- SwiftUI app.
- Auth.
- Chat.
- Streaming.
- Audio playback.
- Image messages.
- Native context.
- Native local actions.
- Proactive tab.
- Connectors.
- Memory.
- History.
- Settings.

Connectors:

- Google.
- Telegram.
- Uber.
- Uber Eats.
- Deliveroo.
- Netflix.
- Trainline.
- Maps.

Not visibly present:

- Pendant firmware.
- BLE bridge.
- Hardware CAD.
- PCB design.
- Dynamic Island.
- Accessibility API.
- Production billing/subscription.
- Payment flow.
- User-facing privacy/legal docs.
- Full production observability.

20. Main Risks

20.1 Latency

Voice assistant quality depends on latency.

Current mitigations:

- Gemini Live prototypes.
- Streaming text.
- Sentence-level TTS.
- Prompt/context caching.
- Deterministic routing.

Still needed:

- Real benchmark numbers.

- Device testing.
- Bad-network behavior.
- Audio interruption/cancel behavior.

20.2 Cost

The new £15/month cap increases pressure.

Need:

- Per-turn cost tracking.
- Model routing.
- TTS duration limits.
- Rate limits.
- Local/native execution first where possible.
- Annual web subscription push.

20.3 Connector Reliability

Risks:

- OAuth expiry.
- API permission changes.
- Deep links changing.
- TransportAPI limitations.
- Telegram session issues.
- Google verification requirements.

Mitigations already present:

- Connector health diagnostics.
- Reconnect states.
- Fallback links.
- Review gating.

20.4 Hardware Feasibility

Risks:

- Bone conduction audibility from pendant body.
- Battery life.
- BLE reliability.
- Acoustic privacy/leakage.
- Waterproofing/sweat.
- Titanium CNC cost.
- Charging alignment.

Must test early:

- Cheap bone conduction transducer on collarbone.
- ESP32-S3 BLE/audio prototype.
- Battery draw.
- Mic quality from pendant position.

20.5 Trust

Risks:

- Always-listening perception.
- Token/data concerns.
- Accidental sends/orders.

Mitigations:

- Press-to-talk.
- No camera.
- Review-gated high-risk actions.
- Visible memory.
- Clear privacy story.

20.6 App Store / Platform

Risks:

- Accessibility automation restrictions.
- Background execution limits.
- Push notification approval/behavior.
- OAuth app verification.
- Deep-link fragility.

Mitigations:

- Native local actions where allowed.
- TestFlight first.
- Clear user-triggered action model.

21. Roadmap

21.1 Immediate

Product/code:

- Verify current backend deploy.
- Run smoke tests in an environment with npm available.
- Confirm `/chat`, `/process-audio`, `/companion-live`, and `/realtime-voice` work against Cloud Run.
- Benchmark current voice path vs Gemini Live.
- Verify Google OAuth end-to-end.
- Verify Telegram auth end-to-end.
- Verify Maps/Trainline fallbacks.
- Verify proactive Cloud Run Job schedule.
- Update pricing everywhere to max £15/month.

iOS:

- Build and run SwiftUI app.
- Confirm auth.
- Confirm SSE chat.
- Confirm image chat.
- Confirm local actions.
- Confirm HealthKit/location context sync.
- Confirm briefings.

Business:

- Rewrite economics around £14.99.
- Decide annual price.
- Decide Kickstarter subscription inclusion.

21.2 June–July

Hardware:

- ESP32-S3 prototype.
- Mic test.
- Bone conduction test.
- BLE phone bridge prototype.
- Resin shell/physical dummy.

Software:

- Add BLE bridge to iOS.
- Add live voice path to native app if not already done.
- Add basic pendant event simulation.
- Add production logging and metrics.

21.3 August

TestFlight:

- 50–100 users.
- Daily-use loop.
- Proactive briefings.
- Native actions.
- Memory.
- Connector setup.
- Feedback collection.

Marketing:

- Demo script.
- Landing/Kickstarter page.
- Product renders.
- Press story.

21.4 September–December

Kickstarter:

- Launch.
- Convert TestFlight users.
- Hardware sample.
- PCBA quotes.
- CNC sample.
- Manufacturing plan.

Production:

- nRF52840 PCB.
- KiCad design.
- JLCPCB PCBA.
- Titanium supplier.
- Charger prototype.

21.5 2027

Scale:

- Fulfil Kickstarter.
- Subscription conversion.
- Shell drops.

- Creator affiliates.
- Angel round only from strength.
- On-device intelligence.
- Deeper native automation.

22. Key Decisions To Make

1. Final subscription price:
 - Recommended: £14.99/month and £129-£149/year.
2. Teal vs warm stone accent:
 - Current app leans warm. May doc says teal. Pick one system.
3. Gemini Live primary path:
 - Decide only after benchmark and device testing.
4. Uber strategy:
 - Current code uses deep links/fallbacks. If REST booking is still desired, that is future work.
5. MCP strategy:
 - Keep separate as external-agent bridge, or merge useful tools into main connector system.
6. Native-first vs cloud-first:
 - iOS code already has strong native local action capability. Leaning into this reduces cost and latency.
7. Privacy/token hardening:
 - Must be addressed before real users.
8. Hardware claim discipline:
 - Do not overclaim bone conduction or battery life until tested.

23. One-Sentence Current State

Oxy has evolved from a product concept with a backend into a real multi-surface assistant stack: authenticated Cloud Run backend, PWA, SwiftUI iOS app, Gemini voice/chat/image layer, memory, proactive briefings, native context, connector actions, and safety reviews, while the pendant hardware, BLE bridge, manufacturing, billing, and production privacy hardening remain the major unfinished pillars.